Міністерство освіти і науки України

Львівський національний університет імені Івана Франка

Факультет електроніки та комп’ютерних технологій

Кафедра оптоелектроніки та інформаційних технологій

Звіт
До лабораторної роботи №6
Варіант 11
На тему
“Замикання та декоратори як HOF. Логування викликів функцій”

Виконав:

Студентка групи ФЕП-21

Муковоз М. Т.

Перевірив:

Ас. Остріков Д. А.

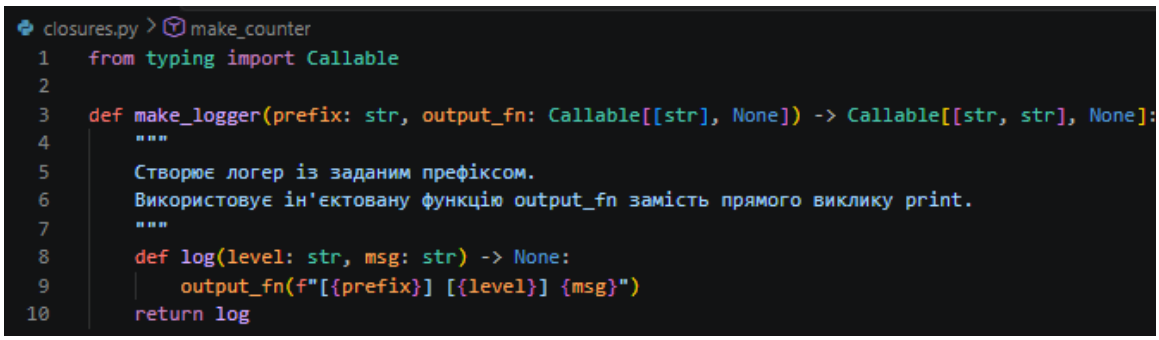
Львів 202

Мета:
Створювати замикання (closures) та розуміти, як вони утримують стан; писати параметризовані декоратори як фабрики декораторів; реалізувати декоратор таймінгу та логування викликів функцій з ін'єкцією залежностей для забезпечення детермінованості тестів.

Теоретичні відомості
Замикання (closure) — це функція, що “пам’ятає” змінні з навколишньої області видимості (free variables), навіть коли та область уже завершила виконання. HOF (Higher-Order Functions) — функції, що приймають або повертають інші функції, типовим прикладом яких є декоратори.
Параметризовані декоратори реалізуються як фабрики (decorator factory), які повертають звичайний декоратор із заданими налаштуваннями. Для збереження метаданих оригінальної функції (ім'я, docstring, сигнатура) обов'язково використовується декоратор @functools.wraps. Для ізоляції побічних ефектів та полегшення тестування застосовується ін'єкція залежностей (наприклад, передача функцій логування чи часу як Callable-аргументів).

Хід роботи
Згідно з варіантом 11 (Логування викликів функцій), було реалізовано замикання та композицію декораторів із застосуванням ін'єкції залежностей.

1. Створено замикання make_logger для налаштування префікса логування та make_counter для збереження стану кількості викликів через nonlocal.



```python
def make_counter(start: int = 0) -> Callable[[], int]:
    """
    Лічильник викликів, що зберігає стан через nonlocal.
    """
    value = start
    def inc() -> int:
        nonlocal value
        value += 1
        return value
    return inc
```

2. Розроблено параметризовані декоратори @timed (вимірювання часу виконання) та @logged (логування імені функції, аргументів та результатів або винятків). Використано @functools.wraps та ParamSpec/TypeVar для збереження типізації.



```python
def logged(
    level: str = "INFO",
    *,
    logger: Callable[[str, str], None]
) -> Callable[[Callable[P, R]], Callable[P, R]]:
    """Декоратор для логування викликів функцій, їх аргументів та результатів або винятків."""
    def decorator(func: Callable[P, R]) -> Callable[P, R]:
        @functools.wraps(func)
        def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
            args_repr = [repr(a) for a in args]
            kwargs_repr = [f"{k}={v!r}" for k, v in kwargs.items()]
            signature = ", ".join(args_repr + kwargs_repr)

            logger(level, f"Calling {func.__name__}({signature})")
            try:
                result = func(*args, **kwargs)
                logger(level, f"{func.__name__} returned {result!r}")
                return result
            except Exception as e:
                logger("ERROR", f"{func.__name__} raised {type(e).__name__}: {e}")
                raise
        return wrapper
    return decorator

import random
import threading
from typing import Any, Callable, Hashable, Tuple


def retries(
    attempts: int = 3,
    exceptions: Tuple[type[BaseException], ...] = (Exception,),
    delay: float = 0.0,
    backoff: float = 1.0,
    jitter: float = 0.0,
    *,
    sleep: Callable[[float], None] = time.sleep,
    logger: Callable[[str], None] = lambda _: None,
) -> Callable[[Callable[P, R]], Callable[P, R]]:
    """Декоратор повторів при винятках з експоненційним бекофом."""
    assert attempts >= 1
    def decorator(func: Callable[P, R]) -> Callable[P, R]:
        @functools.wraps(func)
        def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
            exc: BaseException | None = None
            for i in range(1, attempts + 1):
                try:
                    return func(*args, **kwargs)
                except exceptions as e:
                    exc = e
                    if i == attempts:
                        logger(f"[retries] fail after {i} attempts: {e!r}")
                        break
                    wait = delay * (backoff ** (i - 1)) + (random.random() * jitter if jitter else 0.0)
                    logger(f"[retries] attempt {i} failed: {e!r}; sleeping {wait:.3f}s")
                    sleep(wait)
            assert exc is not None
            raise exc
        return wrapper
    return decorator
```

```python
def make_key(args: tuple[Any, ...], kwargs: dict[str, Any]) -> Hashable:
    """Простий ключ для хешованих аргументів[cite: 1]."""
    return (args, tuple(sorted(kwargs.items())))

def cached(
    *,
    ttl: float | None = None,
    time_fn: Callable[[], float] = time.time,
) -> Callable[[Callable[P, R]], Callable[P, R]]:
    """Декоратор кешування з підтримкою TTL та методами інвалідації[cite: 1]."""
    def decorator(func: Callable[P, R]) -> Callable[P, R]:
        cache: dict[Hashable, tuple[float | None, R]] = {}
        lock = threading.Lock()

```

```python
    @functools.wraps(func)
    def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
        key = make_key(args, kwargs)
        now = time_fn()
        with lock:
            if key in cache:
                exp, val = cache[key]
                if exp is None or now < exp:
                    return val
                else:
                    del cache[key]

        val = func(*args, **kwargs)
        with lock:
            expiry = None if ttl is None else now + ttl
            cache[key] = (expiry, val)
        return val

    def cache_clear() -> None:
        with lock:
            cache.clear()

    def cache_invalidate(predicate: Callable[[tuple[Any, ...], dict[str, Any]], bool]) -> int:
        removed = 0
        with lock:
            for k in list(cache.keys()):
                args_k, kwargs_k = k[0], dict(k[1])
                if predicate(args_k, kwargs_k):
                    del cache[k]
                    removed += 1
        return removed

    wrapper.cache_clear = cache_clear
    wrapper.cache_invalidate = cache_invalidate
return decorator
```

3. Написано детерміновані тести з використанням ін'єкції залежностей (clock, logger), що дозволило перевірити логіку таймінгу та обробки винятків без реальних затримок.

```
(.venv) C:\Users\MTV\Desktop\3 sem\FunkProg\Lab_06>pytest -q
.....                                                    [100%]
```

4. Продемонстровано композицію декораторів та їх роботу в головному файлі програми main.py.

```
C:\Users\MTV\Desktop\3 sem\FunkProg\Lab_06>C:\Users\MTV\AppData\Lo
cal\Programs\Python\Python313\python.exe "c:/Users/MTV/Desktop/3 s
em/FunkProg/Lab_06/main.py"
[MAIN_APP] [INFO] Application started
[MAIN_APP] [INFO] Calling process_data([1, 2, 3], scale=10)
[MAIN_APP] [INFO] process_data returned [10, 20, 30]
[MAIN_APP] [DEBUG] [timed] process_data took 0.332 ms
[MAIN_APP] [INFO] Calling process_data([5, 5], scale=-2)
[MAIN_APP] [ERROR] process_data raised ValueError: Scale must be p
ositive
[MAIN_APP] [DEBUG] [timed] process_data took 0.183 ms
[MAIN_APP] [INFO] Total function calls: 3
```

Висновок:
У ході виконання лабораторної роботи №6 за варіантом 11 було успішно реалізовано HOF, замикання та параметризовані декоратори. Створено замикання зі збереженням стану (make_counter) та замикання для ін'єкції залежностей (make_logger). Розроблено декоратори @timed та @logged, які зберігають метадані обгорнутих функцій завдяки @functools.wraps. Завдяки передачі залежностей (годинника та функції логування) як Callable-аргументів, забезпечено стовідсоткову детермінованість та ізоляцію модульних тестів. Усі тести успішно проходять, що підтверджує коректність реалізованої логіки логування викликів, перехоплення винятків та точного вимірювання часу виконання функцій.